

HOMEWORK 4 SUBMISSION

Use this template to record your answers for Homework 4. Add your answers using L^AT_EX and then save your document as a PDF to upload to Gradescope. You are required to use this template to submit your answers. **You should not alter this template in any way** other than to insert your solutions. You must submit all **11** pages of this template to Gradescope. Do not remove the instructions page(s). Altering this template or including your solutions outside of the provided boxes can result in your assignment being graded incorrectly.

You should also export your code as a .py file and upload it to the **separate** Gradescope coding assignment. Remember to mark all teammates on **both** assignment uploads through Gradescope.

Instructions for Specific Problem Types

On this homework, you must fill in blanks for each problem. Please make sure your final answer is fully included in the given space. **Do not change the size of the box provided.** For short answer questions you should **not** include your work in your solution. Only provide an explanation or proof if specifically asked.

Fill in the blank: What is the course number?

10-703

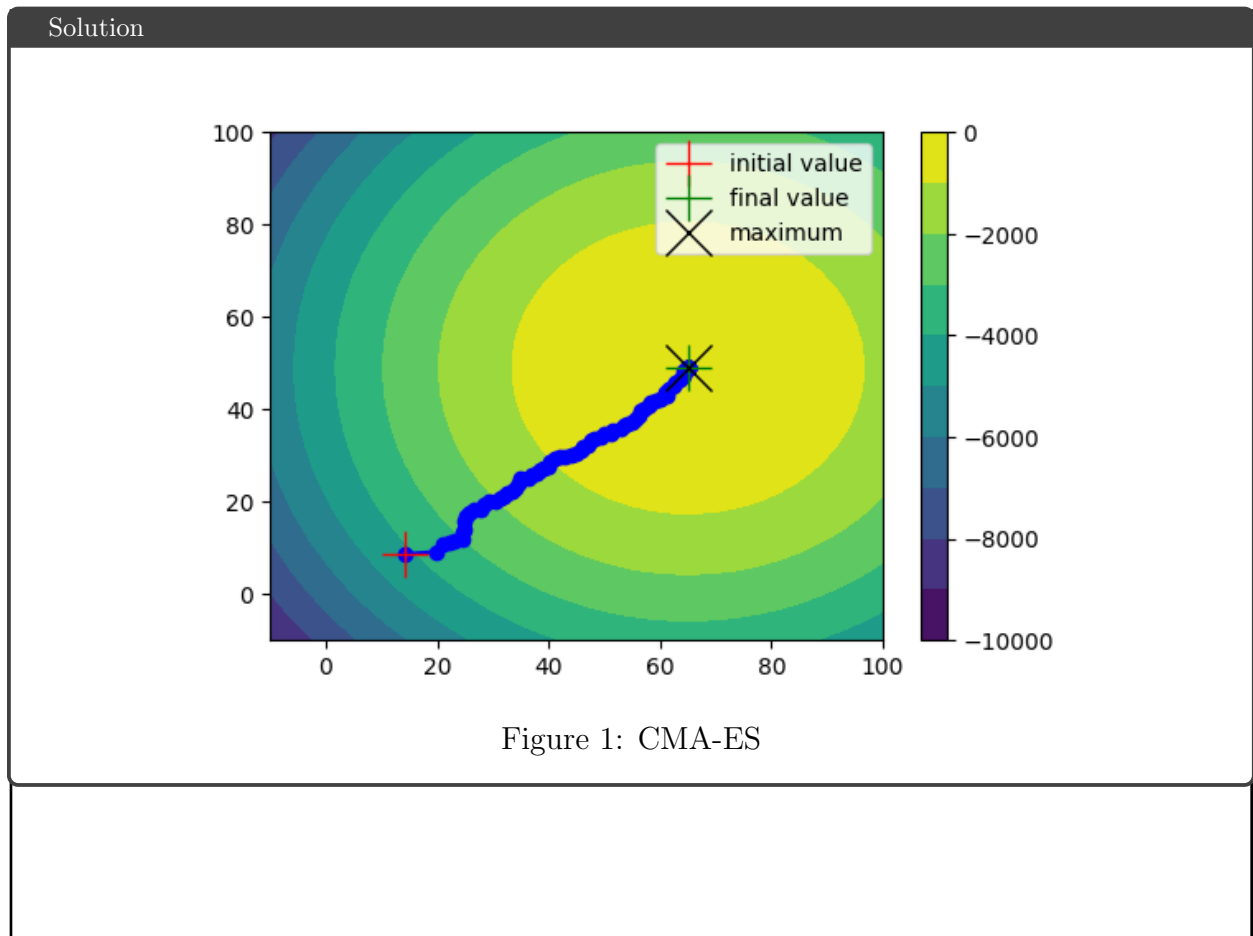
Problem 0: Collaborators

Enter your team members' names and Andrew IDs in the boxes below. If you worked in a team with fewer than three people, leave the extra boxes blank.

Name 1:	Ruiyang Zhou	Andrew ID 1:	ruiyangz
Name 2:	Sangram kumar saho	Andrew ID 2:	Sangrams
Name 3:		Andrew ID 3:	

Problem 1: CMA-ES (24 pts)

1.1 Plot of CMA-ES on simple objective function (10 pts)



1.2 RL reward of fixed policies (4 pts)

$x = (-1, -1, -1, -1, -1) :$	15.6
$x = (1, 0, 1, 0, 1) :$	14.3
$x = (0, 1, 2, 3, 4) :$	9.4

1.3 Plot of CMA-ES on Cartpole (10 pts)

Solution

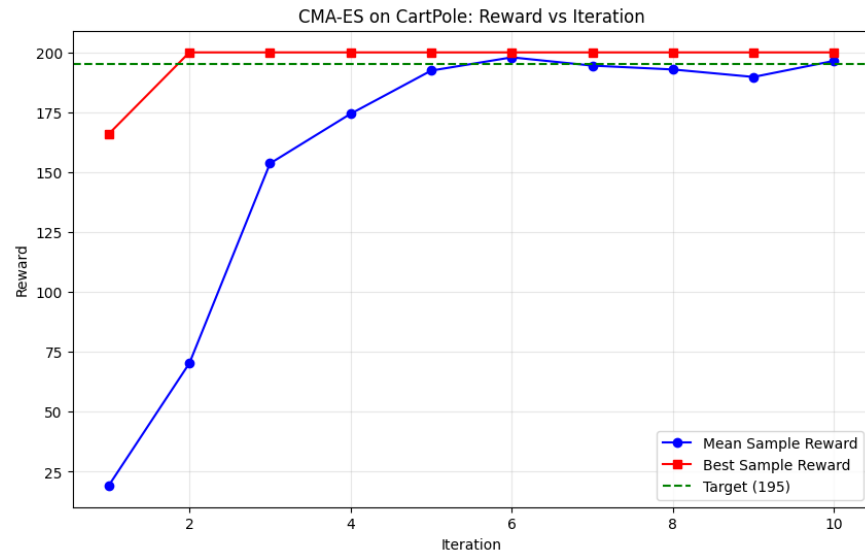
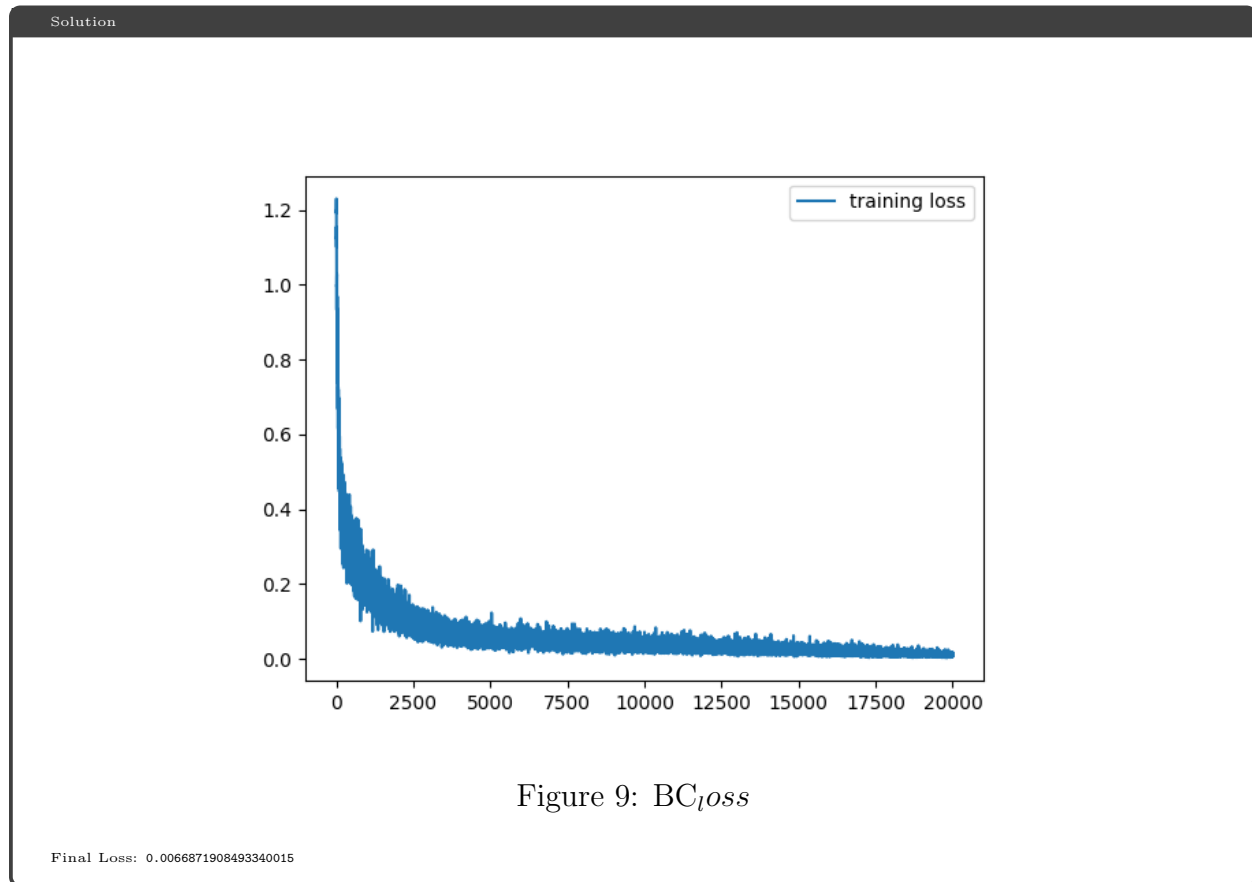


Figure 2: Cartpole

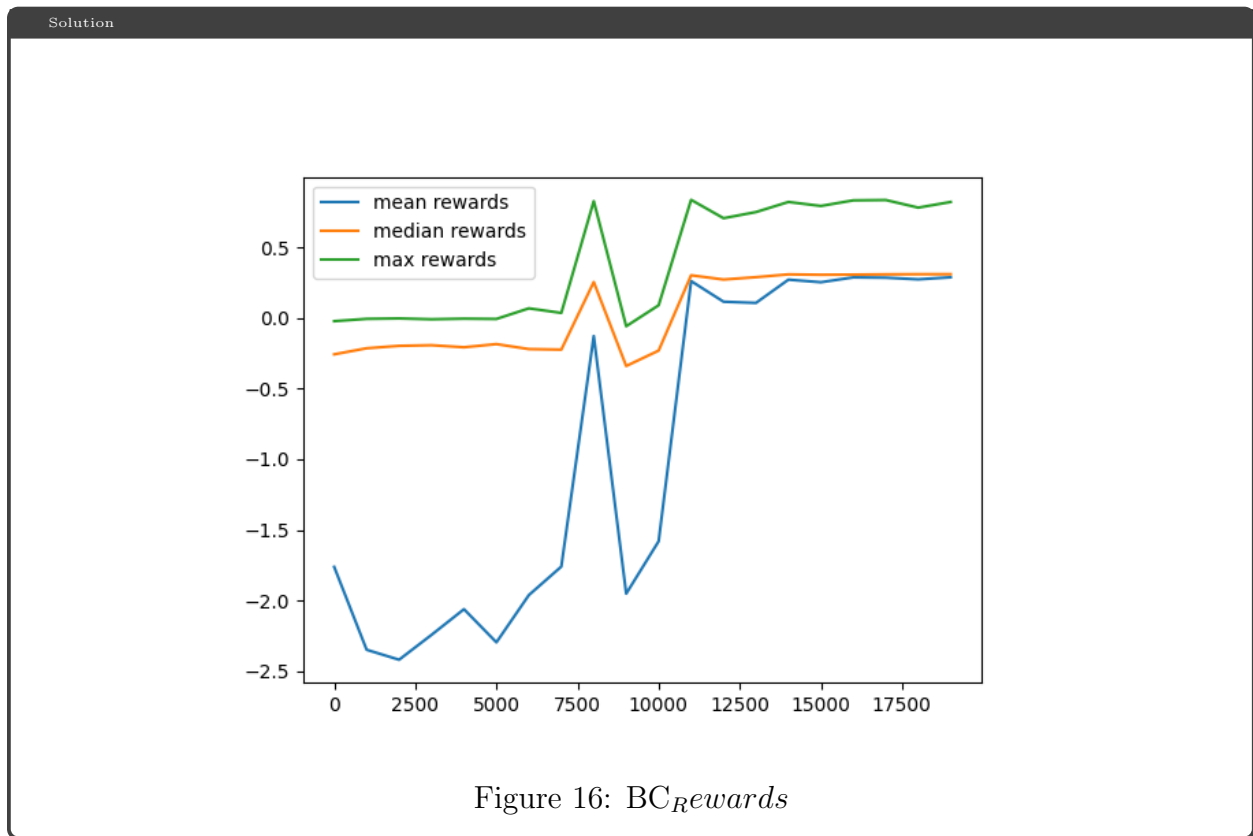
Problem 2: Imitation Learning (62 pts)

Problem 2.1: BC (14 pts)

2.1.1 Loss Plot + Final Loss Value BC (6 pts)



2.1.2 Rewards Plot BC (6 pts)



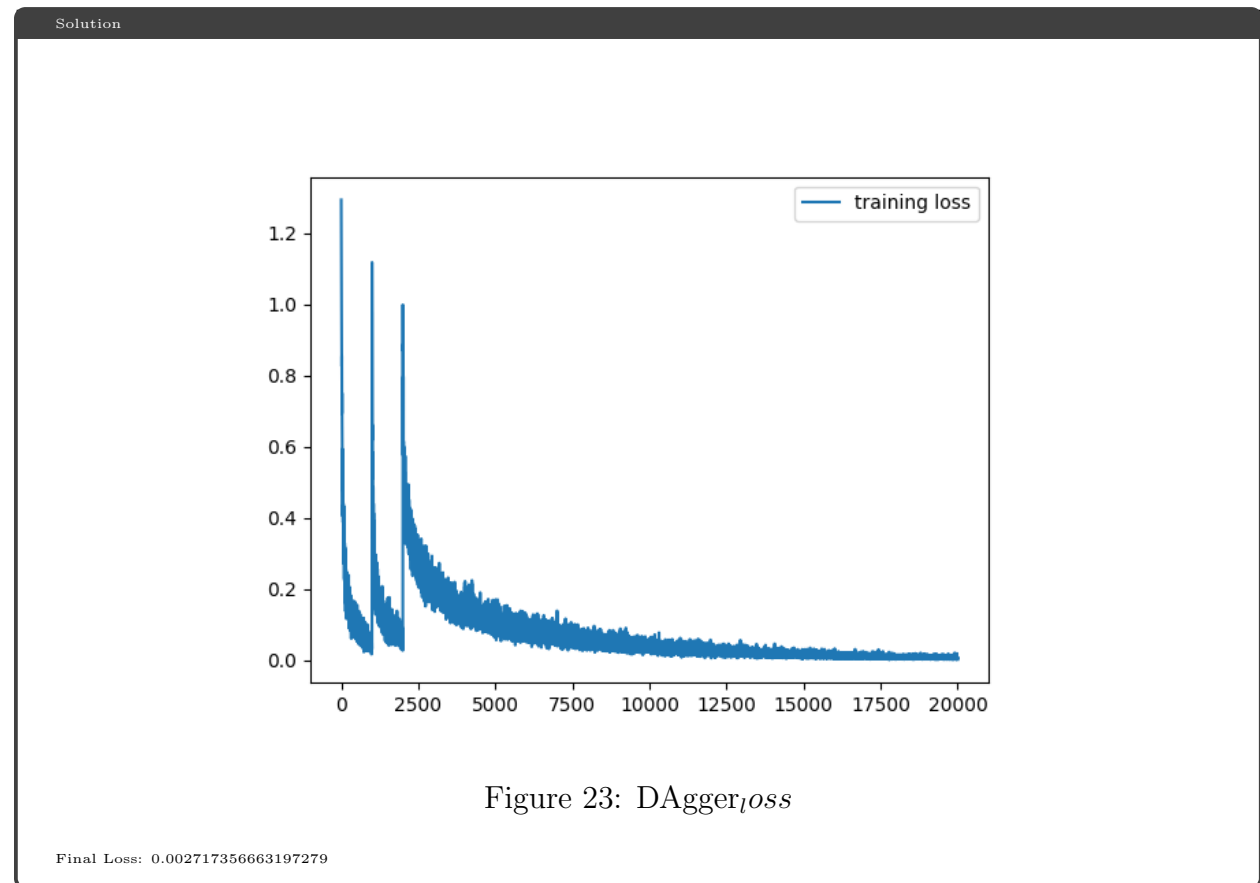
2.1.3 GIF link BC (2 pts)

Solution

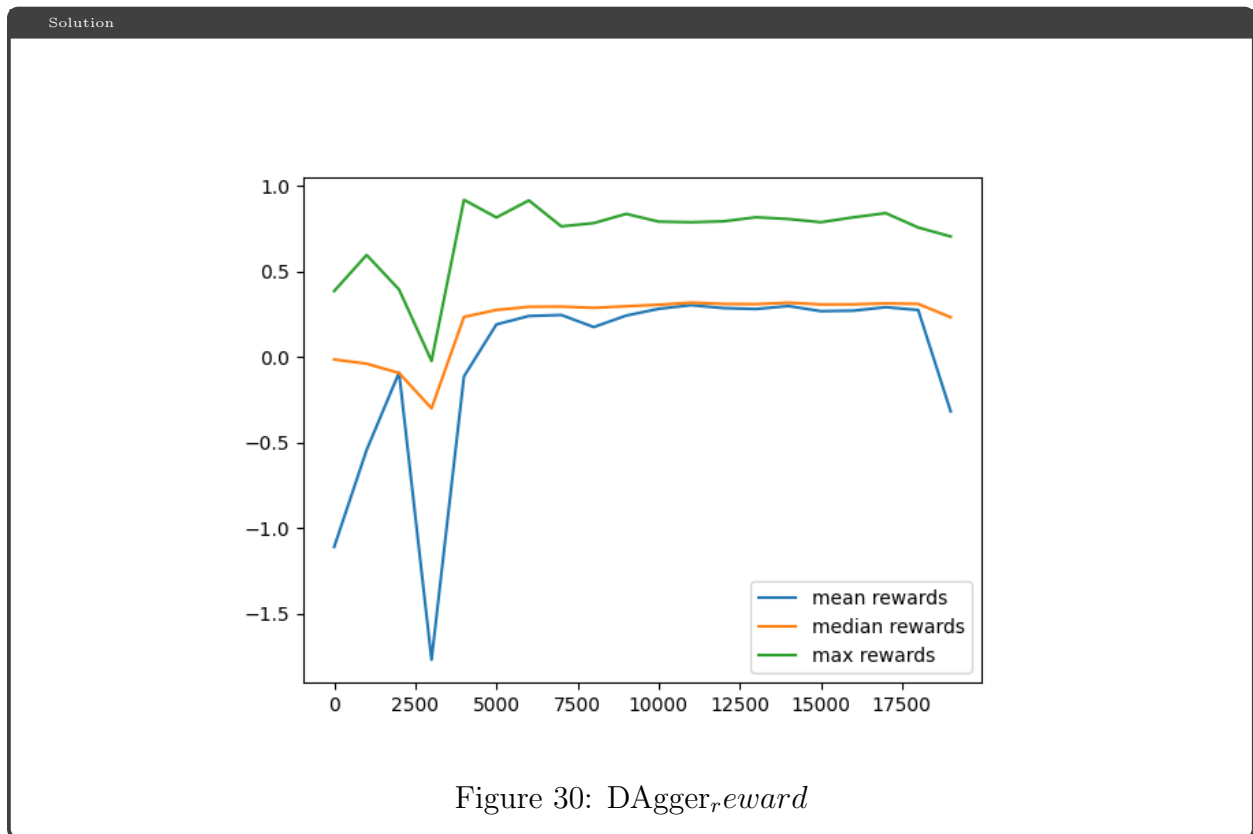
https://drive.google.com/file/d/1zqWvJ-BsJvdY8WeBPYgbmOD2KVtHx/view?usp=drive_link

Problem 2.2: DAgger (18 pts)

2.2.1 Loss Plot DAgger (6 pts)



2.2.2 Rewards Plot DAgger (6 pts)



2.2.3 GIF link DAgger (2 pts)

Solution

https://drive.google.com/file/d/1LZ1p0M_sFj31rMHUkC1bbSufla1V7pKm/view?usp=sharing

2.2.4 Compare DAgger training with BC (written, 4 pts)

Solution

DAgger performed better than BC because it continually collects expert corrections on the states the policy actually visits, closing the distribution gap between training and testing. BC only learns from the expert's demonstration data, so small errors can accumulate and lead to unstable or overly aggressive actions.

Problem 2.3: Diffusion Policy (30 pts)

2.3.1 Loss Plot + Final Loss value Diffusion Policy (6 pts)

Solution

Final Loss: 0.04277658462524414

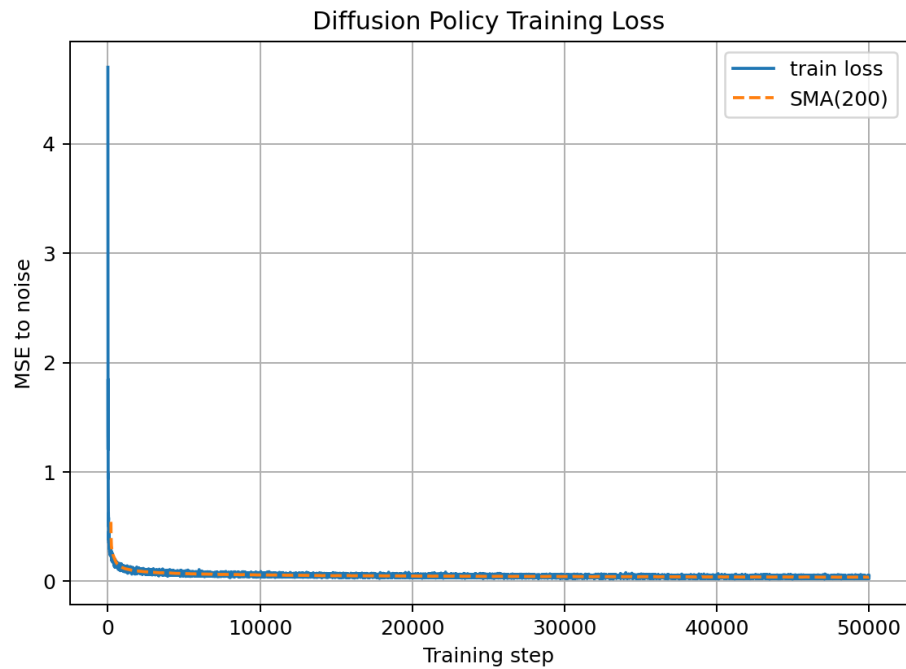


Figure 31: $\text{diffusion}_{\text{training loss}}$

2.3.2 Rewards Diffusion Policy (15 pts)

2.3.2.1; 3 actions evaluated in a row (5 pts)

avg trajectory time: 19.72 s mean: 165.22 median: 253.03 max: 264.24

2.3.2.2; 2 actions evaluated in a row (5 pts)

avg trajectory time: mean: median: max:

2.3.2.3; 1 action evaluated in a row (5 pts)

avg trajectory time: mean: median: max:

2.3.3 GIF link Diffusion Policy (2 pts)

Solution

[https://drive.google.com/file/d/19kRvGgcA_XYxYO4LnzdEWi5VW3zGzQL - /view?usp = sharing](https://drive.google.com/file/d/19kRvGgcA_XYxYO4LnzdEWi5VW3zGzQL/view?usp=sharing)

2.3.4 Compare diffusion policy and simple model runtime (written, 4 pts)

Solution

The first one: Diffusion inference is iterative. Each action prediction requires running the model through dozens of denoising steps, where every step performs a forward pass through a large transformer.

Second one: BC/Dagger policies are direct regressors. They output an action in one forward pass, so inference time is essentially constant per step.

2.3.5 Compare diffusion policy with different actions in a row run-time (written, 3 pts)

Solution

When $\text{num} = 1$, the model must run the full diffusion process (e.g., 30 denoising steps) for every single environment step, so it's the slowest.

With $\text{num} = 2$ or 3 , the model predicts multiple future actions in one diffusion pass, so fewer model calls are needed per trajectory—each call takes longer but total calls are fewer—making average trajectory time shorter as num increases.